

PRIMER PARCIAL PRÁCTICO DE LABORATORIO (CORTE 1)

1. Información General del Examen Práctico

Modalidad: Individual (Sala de Cómputo / IDE de preferencia).

Duración Total: 1 Hora y 40 Minutos (100 minutos exactos).

Dominio de Aplicación: Proyecto de Curso - Banco de Preguntas Saber PRO.

Estructura de Tiempo:

1. Fase 1 (0 a 45 min) - Ensamblado y Depuración (Punto 1): Crear la estructura del proyecto en el IDE, copiar/pegar las clases del código base suministrado y corregir los 4 errores/inconsistencias intencionales hasta lograr que el proyecto compile y ejecute perfectamente.

2. Fase 2 (45 a 90 min) - Requerimiento Individual (Punto 2): Cada estudiante debe implementar una funcionalidad o modificación específica asignada de forma individual (según su número asignado) sobre la solución base que ya funciona.

3. Carga de Evidencia (90 a 100 min) Comprimir el proyecto documentado en .zip y cargarlo en el espacio dispuesto en Google Classroom antes que cierre la tarea, cargar archivo de documentación que haya realizado si requiere con capturas de pantalla de la ejecución exitosa en dónde se evidencie la hora.

2. Objetivos de Evaluación y Temas Cubiertos (Talleres 01 a 06)

Este examen evalúa de forma integrada los conceptos prácticos desarrollados durante las semanas 1 a 6 del laboratorio:

1. Principios SOLID: Separación de Responsabilidades (SRP) e Inversión de Dependencias (DIP) mediante interfaces e inyección de dependencias (Talleres 01 y 02).

2. Arquitectura en Capas: Organización limpia en paquetes (model, pipeline, repository, service, app) (Talleres 02 y 04).

3. Patrón Tuberías y Filtros (Pipes and Filters): Pipeline secuencial de validación estructural de preguntas (HU03) (Taller 05).

4. Pruebas Unitarias con JUnit 5: Verificación automatizada del comportamiento de los componentes del sistema (Talleres 01-05).

3. Punto 1 (45 Minutos - 50% de la Nota): Montaje y Corrección del Código Base

Los estudiantes deben crear un proyecto Java en su IDE llamado exactamente:

lisw2_parcial01_apellidoN (ejemplo: lisw2_parcial01_perezJ)

Estructura de Directorios a Crear

```
lisw2_parcial01_perezJ/  
├── src/  
│   ├── main/  
│   │   └── java/  
│   │       └── co/edu/unicauca/saberpro/  
│   │           ├── Main.java  
│   │           ├── model/  
│   │           │   ├── Pregunta.java  
│   │           │   ├── pipeline/  
│   │           │   │   ├── IFiltroPregunta.java  
│   │           │   │   ├── ValidadorPipeline.java  
│   │           │   │   ├── FiltroEnunciado.java  
│   │           │   │   └── FiltroLongitudEnunciado.java  
│   │           │   ├── repository/  
│   │           │   │   ├── IPreguntaRepository.java  
│   │           │   │   └── PreguntaListRepository.java  
│   │           │   └── service/  
│   │           │       ├── IPreguntaService.java  
│   │           │       └── GestorPreguntaServiceImpl.java  
│   └── test/  
│       └── java/  
│           └── co/edu/unicauca/saberpro/  
│               └── PreguntaServiceTest.java  
└── pom.xml
```

Código Base Suministrado (Copiar en cada archivo)

ATENCIÓN: El código contiene 4 ERRORES / BUGS intencionales que impiden su compilación o correcto funcionamiento. El estudiante debe identificar y corregir dichos errores en la primera hora:

Archivo: `src/main/java/co/edu/unicauca/saberpro/model/Pregunta.java`

```
package co.edu.unicauca.saberpro.model;  
  
import java.util.List;  
import java.util.ArrayList;  
  
public class Pregunta {  
    private int id;  
    private String enunciado;  
    private List<String> opciones;  
    private String respuestaCorrecta;  
    private String estado;  
    private int puntaje;  
  
    public Pregunta(int id, String enunciado, List<String> opciones, String  
respuestaCorrecta, String estado, int puntaje) {  
        this.id = id;  
        this.enunciado = enunciado;  
        this.opciones = (opciones != null) ? opciones : new ArrayList<>();  
    }  
}
```

```
        this.respuestaCorrecta = respuestaCorrecta;
        this.estado = estado;
        this.puntaje = puntaje;
    }

    public int getId() { return id; }
    public String getEnunciado() { return enunciado; }
    public List<String> getOpciones() { return opciones; }
    public String getRespuestaCorrecta() { return respuestaCorrecta; }
    public String getEstado() { return estado; }
    public void setEstado(String estado) { this.estado = estado; }
    public int getPuntaje() { return puntaje; }
    public void setPuntaje(int puntaje) { this.puntaje = puntaje; }
}
```

Archivo: src/main/java/co/edu/unicauca/saberpro/pipeline/IFiltroPregunta.java

```
package co.edu.unicauca.saberpro.pipeline;

import co.edu.unicauca.saberpro.model.Pregunta;

public interface IFiltroPregunta {
    boolean procesar(Pregunta pregunta);
}
```

Archivo: src/main/java/co/edu/unicauca/saberpro/pipeline/FiltroEnunciado.java

```
package co.edu.unicauca.saberpro.pipeline;

import co.edu.unicauca.saberpro.model.Pregunta;

public class FiltroEnunciado implements IFiltroPregunta {
    @Override
    public boolean procesar(Pregunta pregunta) {
        if (pregunta == null || pregunta.getEnunciado() == null) {
            return false;
        }
        return !pregunta.getEnunciado().trim().isEmpty();
    }
}
```

Archivo: src/main/java/co/edu/unicauca/saberpro/pipeline/FiltroLongitudEnunciado.java

```
package co.edu.unicauca.saberpro.pipeline;

import co.edu.unicauca.saberpro.model.Pregunta;

public class FiltroLongitudEnunciado implements IFiltroPregunta {
    @Override
    public boolean procesar(Pregunta pregunta) {
        if (pregunta == null) return false;
        // ERROR 1: Lógica invertida en la condición de longitud mínima
        // La regla exige que el enunciado tenga al menos 15 caracteres para ser
    }
}
```

```
válido.  
    if (pregunta.getEnunciado() == null || pregunta.getEnunciado().length() >=  
15) {  
        return false; // BUG: Retorna false si tiene 15 o más caracteres!  
    }  
    return true;  
}  
}
```

Archivo: src/main/java/co/edu/unicauca/saberpro/pipeline/ValidadorPipeline.java

```
package co.edu.unicauca.saberpro.pipeline;  
  
import co.edu.unicauca.saberpro.model.Pregunta;  
import java.util.ArrayList;  
import java.util.List;  
  
public class ValidadorPipeline {  
    private List<IFiltroPregunta> filtros = new ArrayList<>();  
  
    public void addFiltro(IFiltroPregunta filtro) {  
        filtros.add(filtro);  
    }  
  
    public boolean ejecutar(Pregunta pregunta) {  
        for (IFiltroPregunta filtro : filtros) {  
            if (!filtro.procesar(pregunta)) {  
                System.out.println("Validación falló en: " +  
filtro.getClass().getSimpleName());  
                return false;  
            }  
        }  
        return true;  
    }  
}
```

Archivo: src/main/java/co/edu/unicauca/saberpro/repository/IPreguntaRepository.java

```
package co.edu.unicauca.saberpro.repository;  
  
import co.edu.unicauca.saberpro.model.Pregunta;  
import java.util.List;  
  
public interface IPreguntaRepository {  
    boolean guardar(Pregunta pregunta);  
    List<Pregunta> listar();  
    Pregunta buscarPorId(int id);  
}
```

Archivo:
src/main/java/co/edu/unicauca/saberpro/repository/PreguntaListRepository.java

```
package co.edu.unicauca.saberpro.repository;

import co.edu.unicauca.saberpro.model.Pregunta;
import java.util.ArrayList;
import java.util.List;

public class PreguntaListRepository implements IPreguntaRepository {
    private List<Pregunta> bancoPreguntas = new ArrayList<>();

    @Override
    public boolean guardar(Pregunta pregunta) {

        List<Pregunta> bancoPreguntas = new ArrayList<>();
        return bancoPreguntas.add(pregunta);
    }

    @Override
    public List<Pregunta> listar() {
        return bancoPreguntas;
    }

    @Override
    public Pregunta buscarPorId(int id) {
        return bancoPreguntas.stream()
            .filter(p -> p.getId() == id)
            .findFirst()
            .orElse(null);
    }
}
```

Archivo: src/main/java/co/edu/unicauca/saberpro/service/IPreguntaService.java

```
package co.edu.unicauca.saberpro.service;

import co.edu.unicauca.saberpro.model.Pregunta;
import java.util.List;

public interface IPreguntaService {
    boolean registrarPregunta(Pregunta pregunta);
    List<Pregunta> obtenerTodas();
}
```

Archivo:

src/main/java/co/edu/unicauca/saberpro/service/GestorPreguntaServiceImpl.java

```
package co.edu.unicauca.saberpro.service;

import co.edu.unicauca.saberpro.model.Pregunta;
import co.edu.unicauca.saberpro.pipeline.ValidadorPipeline;
import co.edu.unicauca.saberpro.repository.IPreguntaRepository;
import java.util.List;
```

```
public class GestorPreguntaServiceImpl implements IPreguntaService {
    private final IPreguntaRepository repository;
    private final ValidadorPipeline pipeline;

    public GestorPreguntaServiceImpl(IPreguntaRepository repository,
    ValidadorPipeline pipeline) {
        this.repository = repository;
        this.pipeline = pipeline;
    }

    IPreguntaService
    //
    public boolean procesarYGuardarPregunta(Pregunta pregunta) {
        if (pipeline.ejecutar(pregunta)) {
            return repository.guardar(pregunta);
        }
        return false;
    }

    @Override
    public List<Pregunta> obtenerTodas() {
        return repository.listar();
    }
}
```

Archivo: src/main/java/co/edu/unicauca/saberpro/Main.java

```
package co.edu.unicauca.saberpro;

import co.edu.unicauca.saberpro.model.Pregunta;
import co.edu.unicauca.saberpro.pipeline.*;
import co.edu.unicauca.saberpro.repository.*;
import co.edu.unicauca.saberpro.service.*;
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== BANCO DE PREGUNTAS SABER PRO (GRUPO B) ===");

        IPreguntaRepository repository = new PreguntaListRepository();
        ValidadorPipeline pipeline = new ValidadorPipeline();
        pipeline.addFiltro(new FiltroEnunciado());
        pipeline.addFiltro(new FiltroLongitudEnunciado());

        //
        IPreguntaService service = new GestorPreguntaServiceImpl(repository, null);

        Pregunta p1 = new Pregunta(
            1,
            "¿Cuál es la función principal del patrón Microkernel para agregar
módulos sin alterar el núcleo?",
            Arrays.asList("A. Plugins dinámicos", "B. Monolito rígido", "C. Base de
```

```
datos", "D. Redes"),
    "A. Plugins dinámicos",
    "BORRADOR",
    85
);

boolean exito = service.registrarPregunta(p1);
if (exito) {
    System.out.println(" Pregunta registrada con éxito. Total en banco: " +
service.obtenerTodas().size());
} else {
    System.out.println(" No se pudo registrar la pregunta debido a fallos en
el pipeline.");
}
}
}
```

4. Punto 2 (45 Minutos - 50% de la Nota): Requerimiento Individual por Estudiante

Una vez que la aplicación base esté corregida y funcionando correctamente en la consola, cada estudiante debe resolver ÚNICAMENTE la tarea o filtro asignado a su número correspondiente por el docente (distribuido entre los 18 estudiantes del curso):

Opción / Tarea	Tema / Requerimiento	Descripción de la Tarea Individual
Opción 01 (Est. 1 y 11)	Filtro de Puntaje Válido	Implemente 'FiltroPuntajeValido' en 'pipeline' (implements IFiltroPregunta). Retorna false si el puntaje asignado a la pregunta está fuera del rango [1, 100]. InCLÚYALO en el pipeline.
Opción 02 (Est. 2 y 12)	Repositorio en Archivo (DIP/OCP)	Cree 'PreguntaFileRepositoryImpl' en 'repository' implementando 'IPreguntaRepository'. Simule la persistencia imprimiendo '[Guardado en Disco]'. Reemplácelo en Main.java probando OCP/DIP.
Opción 03 (Est. 3 y 13)	Filtro Frases Prohibidas (HU03)	Implemente 'FiltroExpresionesProhibidas' en 'pipeline'. Rechaza la pregunta (retorna false) si el enunciado contiene expresiones ambiguas como 'Todas las anteriores' o 'Ninguna de las anteriores'.
Opción 04 (Est. 4 y 14)	Consulta por Competencia	Agregue el método 'List<Pregunta> obtenerPorCompetencia(String comp)' en servicio y repositorio. En Main.java, registre preguntas de dos competencias y demuestre la consulta filtrada.

Opción 05 (Est. 5 y 15)	Patrón Observer (Auditoría)	Cree la interfaz 'IAuditorPregunta' con el método 'notificarRegistro(Pregunta p)'. Modifique el servicio para registrar un auditor que imprima una alerta por consola al guardar la pregunta.
Opción 06 (Est. 6 y 16)	Filtro Estado Inicial	Implemente 'FiltroEstadoInicial' en 'pipeline'. Valida que la pregunta nueva ingrese estrictamente en estado 'BORRADOR'. Si tiene otro estado inicial (ej. 'PUBLICADA'), retorna false.
Opción 07 (Est. 7 y 17)	Formateador de Examen	Cree la interfaz 'IFormateadorPregunta' con 'String formatear(Pregunta p)' y la clase 'FormateadorTextolmpl' que imprima el ID, el puntaje y las opciones formateadas con viñetas.
Opción 08 (Est. 8 y 18)	Control de Límite de Banco	Modifique el servicio para limitar el banco a un máximo de 5 preguntas. Si se intenta registrar la 6ta pregunta, debe rechazar el registro y retornar false con un mensaje de advertencia.
Opción 09 (Est. 9)	Filtro Competencia Oficial	Agregue el atributo 'competencia' a Pregunta. Implemente 'FiltroCompetenciaOficial' validando que pertenezca a la lista ('Lectura Crítica', 'Razonamiento Cuantitativo', 'Inglés').
Opción 10 (Est. 10)	Cambio de Estado Controlado	Agregue 'actualizarEstado(int id, String nuevoEstado)' en el servicio. La regla de negocio exige que solo se pueda cambiar a 'PUBLICADA' si el estado actual en el banco es 'EN_REVISION'.

5. Criterios de Evaluación y Rúbrica

Criterio	Descripción de Excelencia	Peso (%)	Nota Max.
Punto 1: Montaje y Depuración (Fase 1)	Crea la estructura correcta de paquetes, copia los archivos y corrige exitosamente los 4 errores intencionales permitiendo que el proyecto compile y ejecute.	50%	2.5
Punto 2: Requerimiento Individual (Fase 2)	Implementa correctamente la funcionalidad o filtro asignado según su opción, respetando los principios SOLID y la arquitectura en capas.	40%	2.0

Pruebas Unitarias / Ejecución en Consola	Demuestra el correcto funcionamiento de la solución en la consola o mediante pruebas unitarias con JUnit.	10%	0.5
---	---	-----	-----